

团队作业3-实现文档

抗日战争胜利 80 周年纪念网站 (Yii2 Advanced / 前后台分离)

0. 项目运行与一些默认约定

这个项目基于 **Yii2 Advanced** 做的（前台 + 后台分开跑、分开写）。Web Server (Apache/Nginx) 把请求交给 PHP，数据用 MySQL 存。开发时我们先用 **Gii** 把“能跑的骨架”快速搭起来，然后再按纪念站这种内容型网站的特点，把审核、软删、图片兜底、前台展示这些补齐。

为了避免前后台写成两套、文件散得找不到人，我们在文档里统一按下面的路径说事：

- 前台控制器： `frontend/controllers/*Controller.php`
 - 后台控制器： `backend/controllers/*Controller.php`
 - 共享模型： `common/models/*.php`
 - 迁移文件： `console/migrations/*.php`
 - 前台视图： `frontend/views/<module>/*`
 - 后台视图： `backend/views/<module>/*`
-

1. 项目目录与入口

1) 目录各自干什么

- `frontend`：负责“展示给用户看”的页面（首页、时间轴、地图、人物、影视、留言、团队介绍）
- `backend`：负责“内容怎么管”的后台（首页入口配置、时间轴维护、战役维护、人物管理、影视管理、留言审核）
- `common`：前后台都要用的东西（ActiveRecord 模型、行为、一些通用方法）
- `console`：迁移脚本、初始化脚本（建表、改字段、补数据）

2) 常用访问入口

- 前台首页： `/site/index`
 - 后台首页： `/site/index`（但要走 backend 的入口域/入口地址）
 - Gii： `/gii`（只在开发环境开）
-

2. 数据库迁移与模型生成

项目里大部分模块都按同一条链路落地：**先把表结构用迁移定死 → 用 Gii 生成 CRUD 骨架 → 再把“内容站需要的治理能力”补齐。**

这样写的好处很实际：迁移保证“结构能复现”、Gii 保证“页面不返工”、补齐逻辑保证“真的能运营起来”。

1) 迁移

我们尽量不做“手动在数据库里建表然后截图证明”的那种方式，因为多人协作时太容易乱。更稳的是把所有结构写进 `console/migrations/`，大家拉下代码后能直接迁移出同样的库。

创建迁移文件：

```
php yii migrate/create create_guestbook_message_table
```

在迁移里写表结构（下面是示例写法，风格和项目保持一致）：

```
public function safeUp()
{
    $this->createTable('{{%guestbook_message}}', [
        'id' => $this->primaryKey(),
        'nickname' => $this->string(100)->notNull()->defaultValue('')-
>comment('昵称'),
        'content' => $this->text()->notNull()->comment('留言内容'),
        'is_approved' => $this->boolean()->notNull()->defaultValue(0)-
>comment('审核状态'),
        'created_at' => $this->integer()->notNull()->defaultValue(0),
        'updated_at' => $this->integer()->notNull()->defaultValue(0),
    ], 'ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci
COMMENT="留言表"');

    $this->createIndex('idx_guestbook_is_approved', '{{%guestbook_message}}',
        'is_approved');
    $this->createIndex('idx_guestbook_created_at', '{{%guestbook_message}}',
        'created_at');
}
```

执行迁移：

```
php yii migrate
```

2) Gii

迁移把表建好之后，下一步就是快速生成 Model / CRUD。Gii 不会替你写业务逻辑，但它把最费时间的样板都铺好（Model、searchModel、Controller、Views），对作业这种多模块项目非常省心。

生成 Model（放 common，前后台共用）

- Table Name: 例如 `guestbook_message`
- Model Class: 例如 `GuestbookMessage`
- Namespace: `common\models`

生成结果：`common/models/GuestbookMessage.php`

生成 CRUD（放 backend，做管理端页面）

- Model Class: `common\models\GuestbookMessage`
- Search Model Class: `backend\models\GuestbookMessageSearch`
- Controller Class: `backend\controllers\GuestbookMessageController`

- View Path: `@backend/views/guestbook-message`

生成结果：后台直接能看到列表、筛选、增删改查页面（GridView + 表单 + view）。

3) 补齐

Gii 生成的更像通用管理系统，但纪念站这种内容型网站通常更在意：**内容能不能管住、能不能下架、能不能回溯**，而前台展示又要“克制、清晰、有叙事”。

我们补齐逻辑时一般分两条线：

后台治理这条线通常会加：

- **访问控制**：后台 controller 加 `AccessControl`，没登录就别进管理页
- **敏感动作限制**：删除/审核这类动作用 `VerbFilter` 强制 POST，避免误触
- **状态治理**：
 - 首页入口类内容用 `is_active` 控制启停
 - 留言这种互动内容用 `is_approved` 做审核闸门
 - 人物这种“纪念性内容”更适合 `deleted_at` 软删，避免硬删导致历史断裂
- **列表更像后台在用**：时间戳格式化、状态字段显示中文（待审核/已通过），必要时加快捷按钮

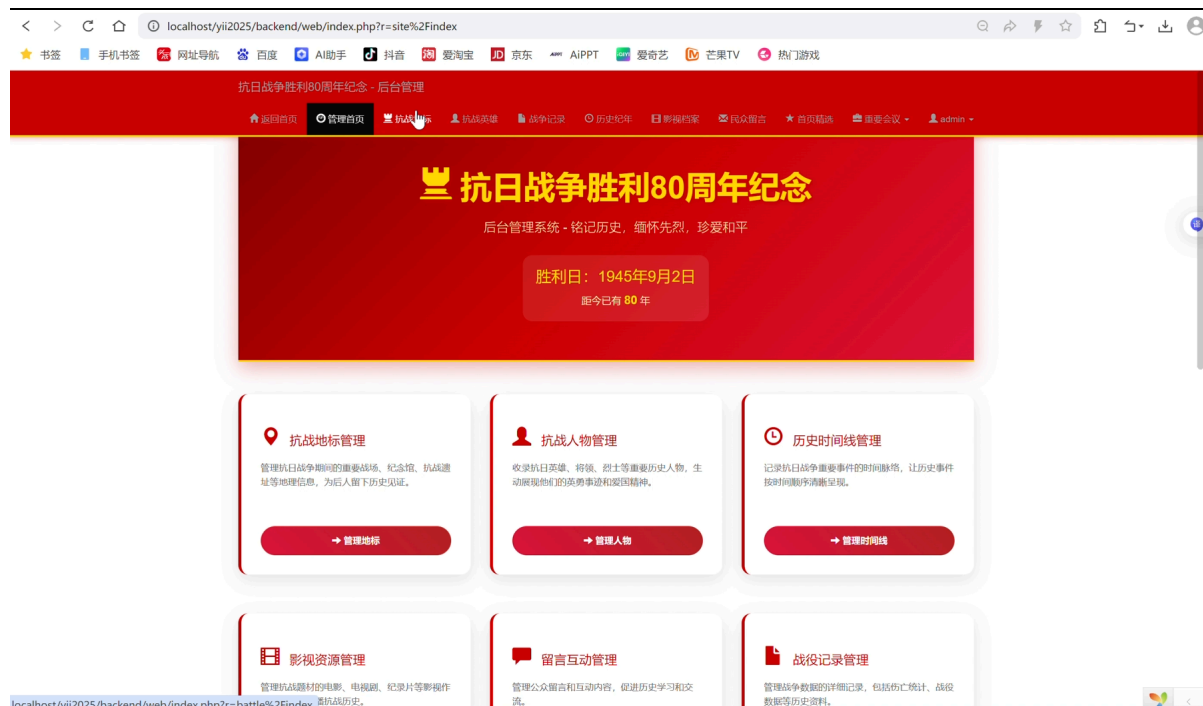
前台体验这条线通常会加：

- **只展示可公开数据**：过滤未启用/未审核/已软删
- **图片兜底**：缺图就给占位图，别让页面裂掉
- **内容分层**：列表页只放摘要，长文放详情页/弹窗，阅读节奏更舒服

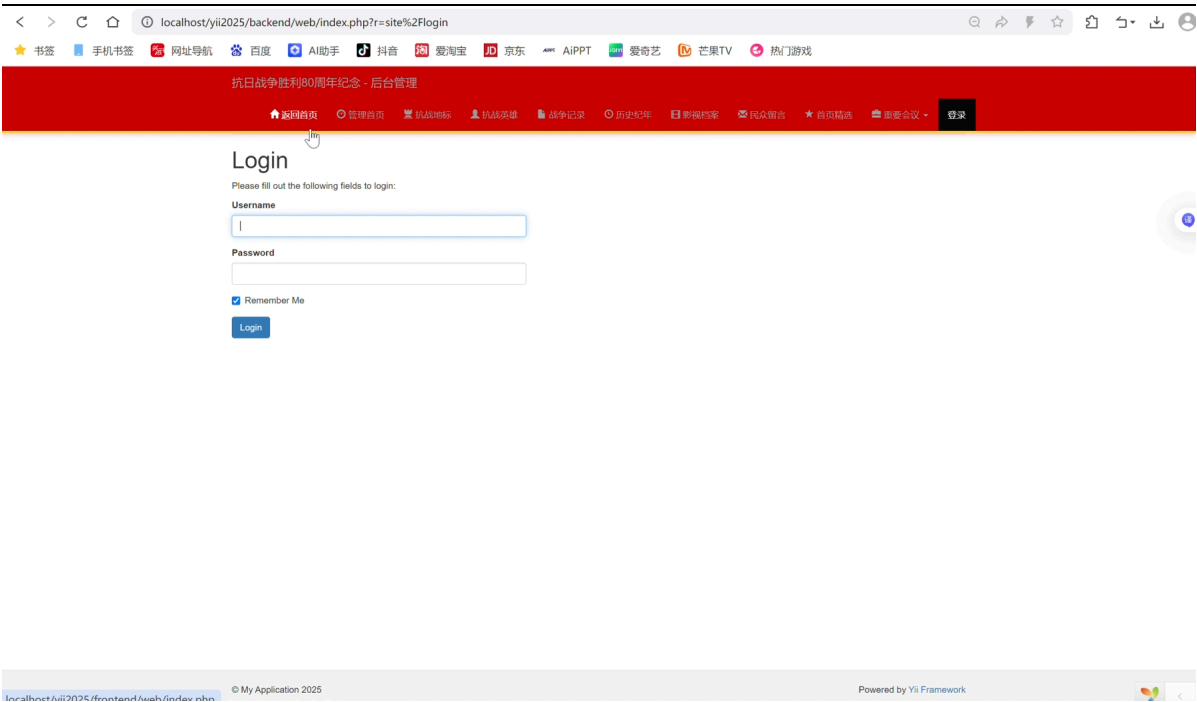
4) 一个很关键的分层点：Model 统一放 common

Advanced 最容易乱的是：同一张表的模型有人放 frontend，有人放 backend，最后变两套规则、两套行为。我们这里统一做法是：**ActiveRecord 都放 common，前后台各写各自的 Controller / Views**。这样验证规则、时间戳行为、软删过滤这些都不会“前台一个样后台一个样”。

整体的后台管理页面如下：



并且需要先登录才能进入后台：



3. 专题首页模块

- 表： `homepage_feature`
- 模型： `common/models/HomepageFeature.php`

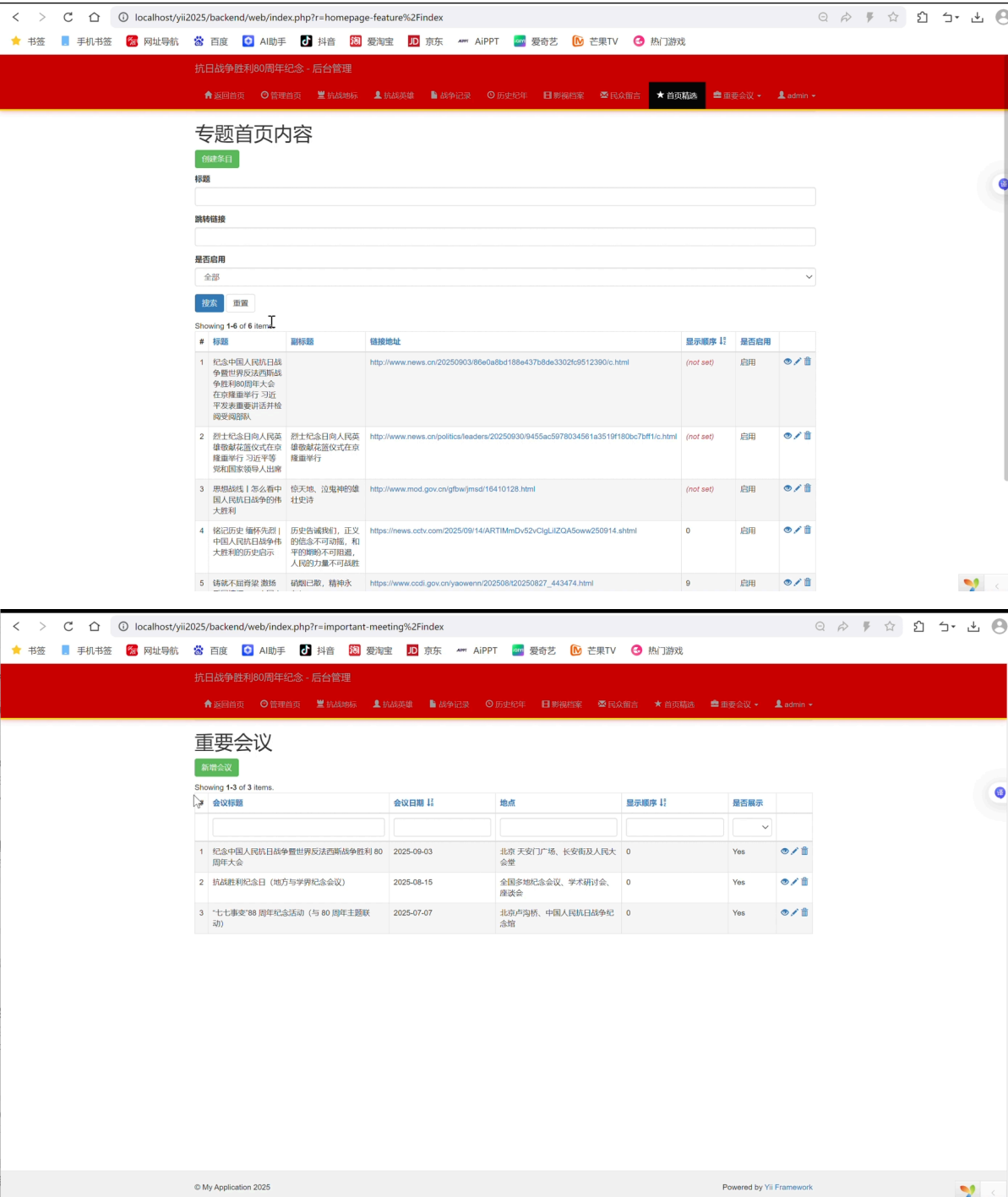
字段大概就是： `title / subtitle / description / image_url / link_url / display_order / is_active / created_at / updated_at`（都是首页入口常见的那套）。

2) 后台实现

- 控制器： `backend/controllers/HomepageFeatureController.php`（Gii 生成后基本可用）
- 视图： `backend/views/homepage-feature/*`

后台主要用来调顺序、启用/停用入口、改图片和跳转链接。首页入口这种东西做成配置化，能少改很多前台代码。

效果如下：



3) 前台实现

- 控制器: `frontend/controllers/SiteController.php`

```
<?php
namespace frontend\controllers;

use yii\web\Controller;
use common\models\HomepageFeature;

class SiteController extends Controller
{
    public function actionIndex()
    {
        $features = HomepageFeature::find()
            ->where(['is_active' => 1])
```

```
->orderBy(['display_order' => SORT_ASC])
->all();

return $this->render('index', [
    'features' => $features,
]);
}
}
```

- 视图: `frontend/views/site/index.php`

渲染时一般会把第一条做“主卡片”，其它做“入口速览”。图片和链接统一通过辅助函数拼 URL（尤其是子目录部署时，不然路径很容易翻车）。

最终效果如下：



4. 时间轴模块

1) 数据库与模型

- 表: `timeline_event`
- 模型: `common/models/TimelineEvent.php`

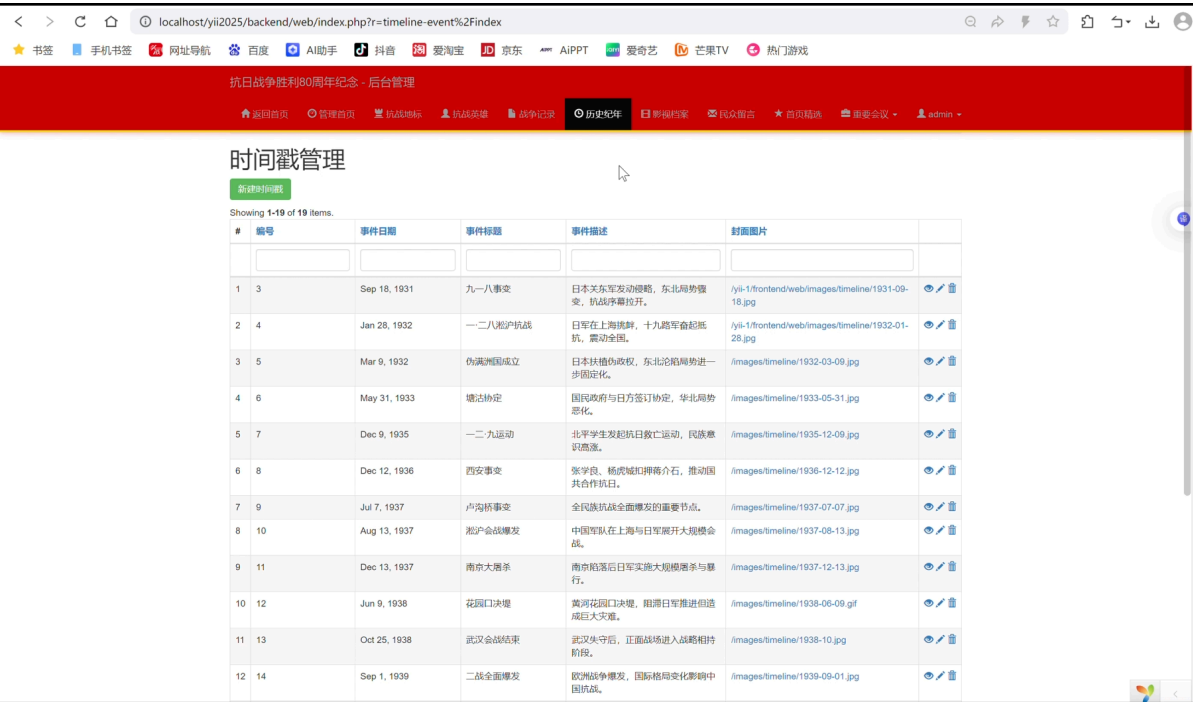
字段: `event_date / title / description / cover_image_url / importance / sort_order / status / created_at / updated_at`。

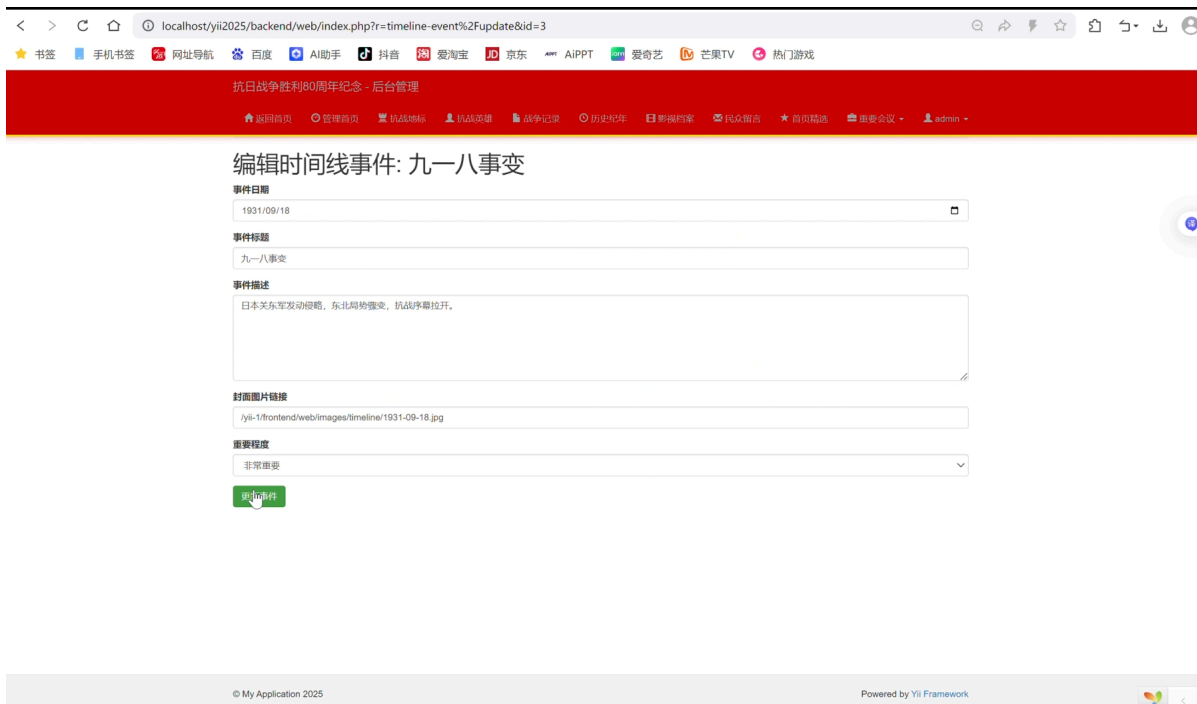
2) 后台实现

- 控制器: `backend/controllers/TimelineEventController.php` (Gii 生成)
- 视图: `backend/views/timeline-event/*`

后台负责录入事件、封面、重要性、展示状态等。

效果如下：





3) 前台实现

- 控制器: `frontend/controllers/TimelineController.php`

```
<?php
namespace frontend\controllers;

use yii\web\Controller;
use common\models\TimelineEvent;

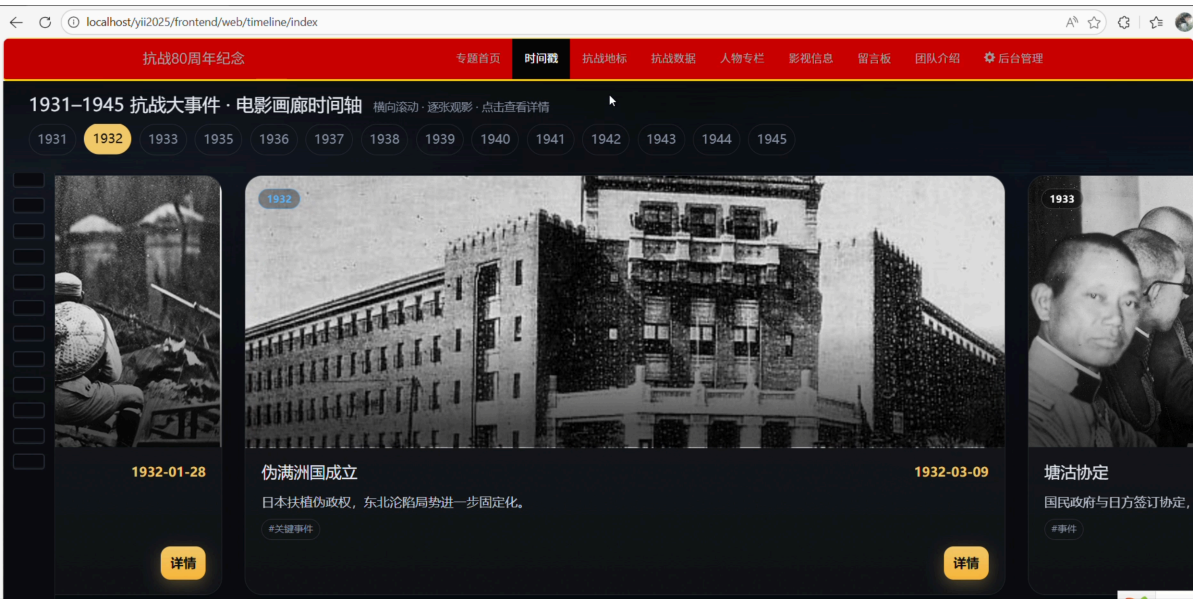
class TimelineController extends Controller
{
    public function actionIndex()
    {
        $events = TimelineEvent::find()
            ->where(['status' => 1])
            ->orderBy(['event_date' => SORT_ASC, 'sort_order' => SORT_DESC])
            ->all();

        return $this->render('index', [
            'events' => $events,
        ]);
    }
}
```

- 视图: `frontend/views/timeline/index.php`

常见做法是把 `$events` 序列化成前端能直接用的数据 (比如挂到 `window.TIMELINE_EVENTS`)，然后做横向滚动、年份跳转 (chips)、弹窗详情展示。封面缺失用占位图，摘要缺失就显示“暂无简介”，别让 UI 空一块。

最终效果如下：



5. 战役地标模块

1) 数据库与模型

- 表: `battle`
- 模型: `common/models/Battle.php`

关键字段: `name / start_date / end_date / main_location / main_latitude / main_longitude / description / result / casualties_*`。

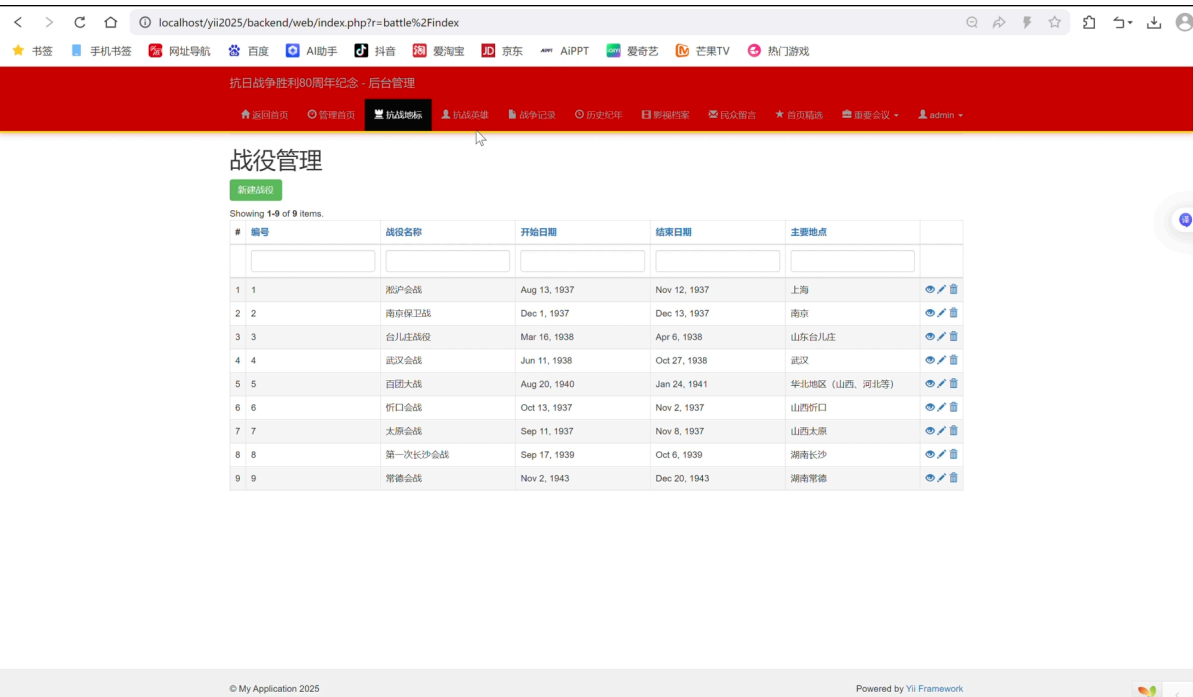
地图能不能画出来, 最核心就是经纬度; 详情页则靠描述、结果、伤亡等字段撑内容。

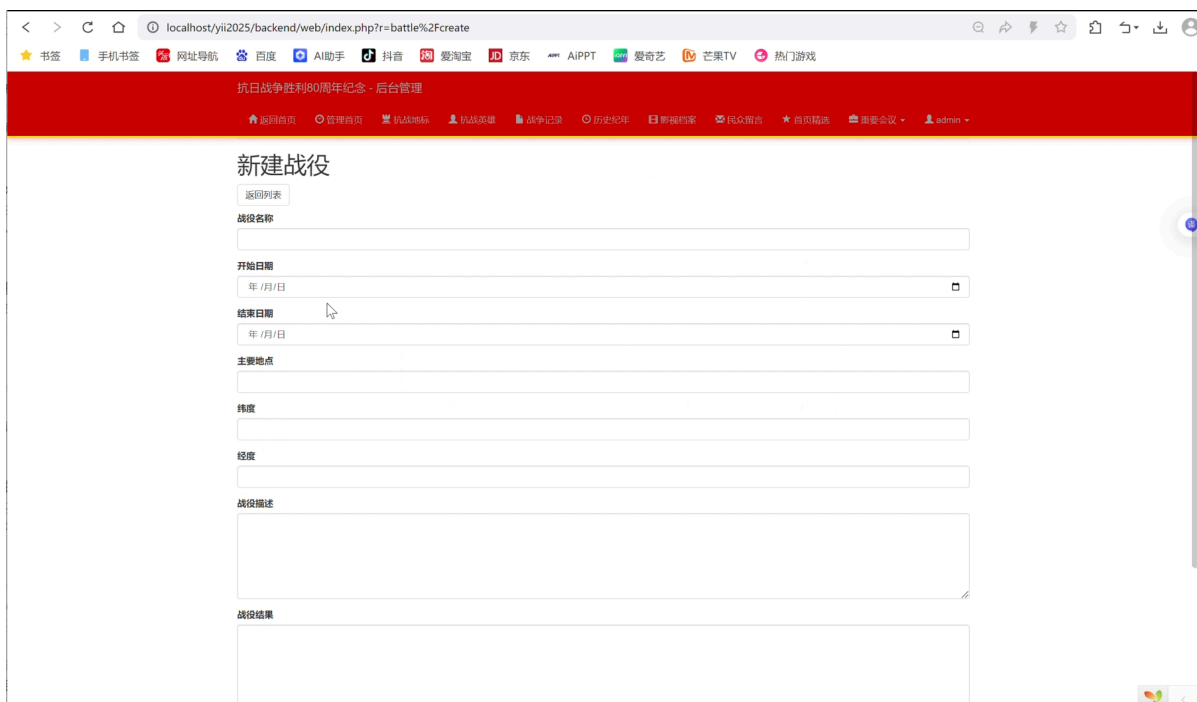
2) 后台实现

- 控制器: `backend/controllers/BattleController.php` (Gii 生成)

后台把经纬度录入齐了, 前台地图就能自动渲染点位, 不需要再单独维护一张“地标表”。

效果如下:





3) 前台实现

- 控制器: `frontend/controllers/BattleController.php`

```
<?php
namespace frontend\controllers;

use yii\web\Controller;
use common\models\Battle;

class BattleController extends Controller
{
    public function actionIndex()
    {
        $battles = Battle::find()
            ->andWhere(['not', ['main_latitude' => null]])
            ->andWhere(['not', ['main_longitude' => null]])
            ->all();

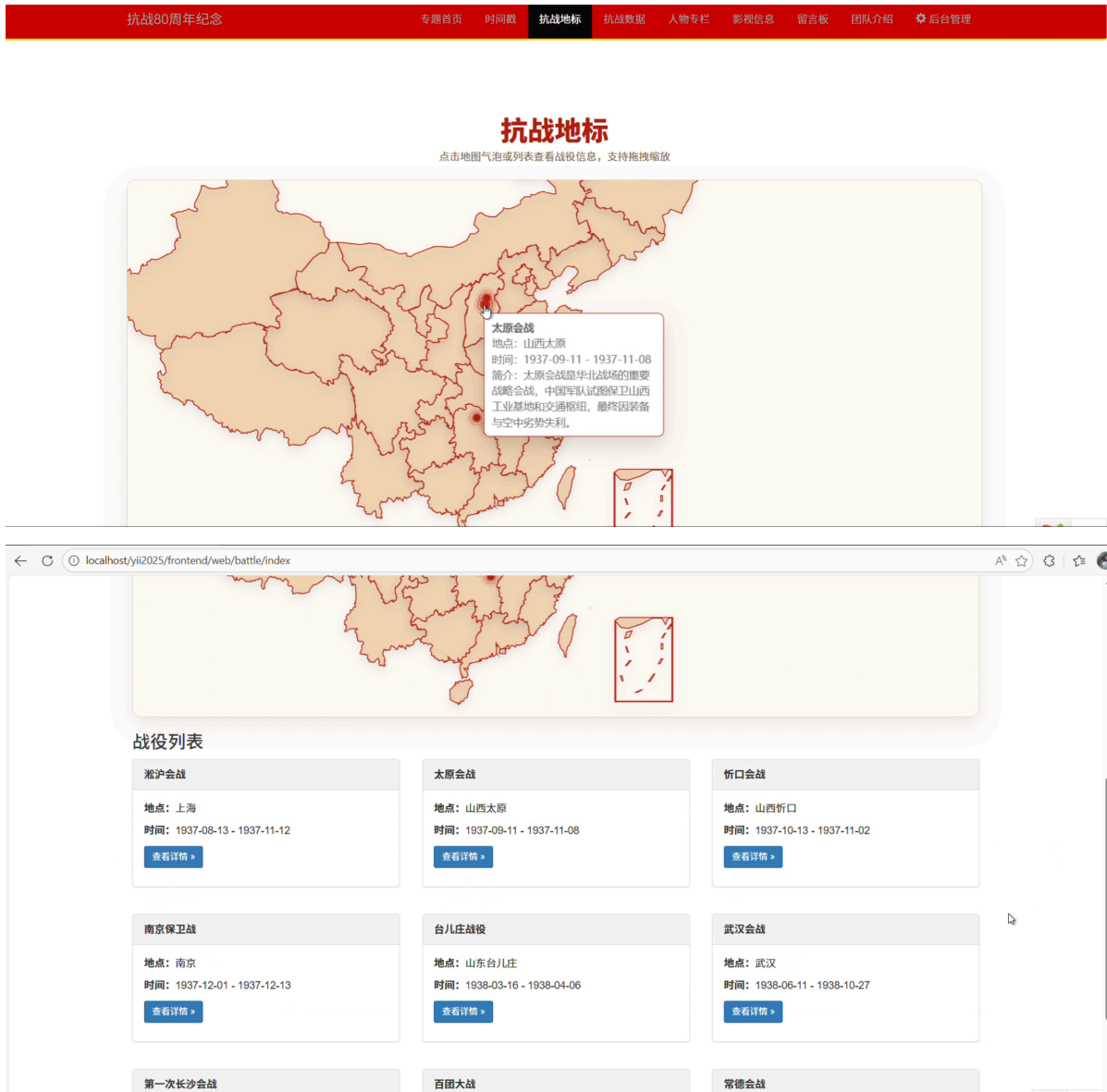
        return $this->render('index', [
            'battles' => $battles,
        ]);
    }

    public function actionView($id)
    {
        $model = Battle::findOne($id);
        if (!$model) {
            throw new \yii\web\NotFoundException('战役不存在');
        }
        return $this->render('view', ['model' => $model]);
    }
}
```

- 视图: `frontend/views/battle/index.php`

地图用 **本地 ECharts + 本地中国地图 GeoJSON** (比如 `frontend/web/js/echarts.min.js` + `frontend/web/js/china.json`)。注册地图后用 `effectScatter` 标点, `tooltip` 展示摘要, 点击跳详情页。

最终效果如下:



6. 抗战数据模块

1) 数据库与模型

- 表: `war_dataset`、`war_record`
- 模型: `common/models/warDataset.php`、`common/models/warRecord.php`


```
war_dataset` 字段典型是: `key / name / source_file / description / display_order /
is_active / created_at / updated_at`
`war_record` 字段固定为: `dataset_id / row_index / data_json / created_at /
updated_at
```

这里的关键点是: `data_json` 不是“偷懒”, 而是**为了让不同数据集字段可以自由变化** (有的有 `year`, 有的有 `period_start/end`, 有的甚至是人物列表/战役列表这种结构), 但表结构仍然稳定。

2) 后台实现控制器:

- `backend/controllers/WarDatasetController.php`
- `backend/controllers/WarRecordController.php`
- 视图:
 - `backend/views/war-dataset/*`
 - `backend/views/war-record/*`

```
INSERT INTO `war_record` (`id`, `dataset_id`, `row_index`, `data_json`,
`created_at`, `updated_at`)
VALUES (198, 1, 2, '{"dataset": "中国伤亡与损失估计 (多来源)
", "period_start": "1937", "period_end": "1945", ...}', 1766406452,
1766411199);
```

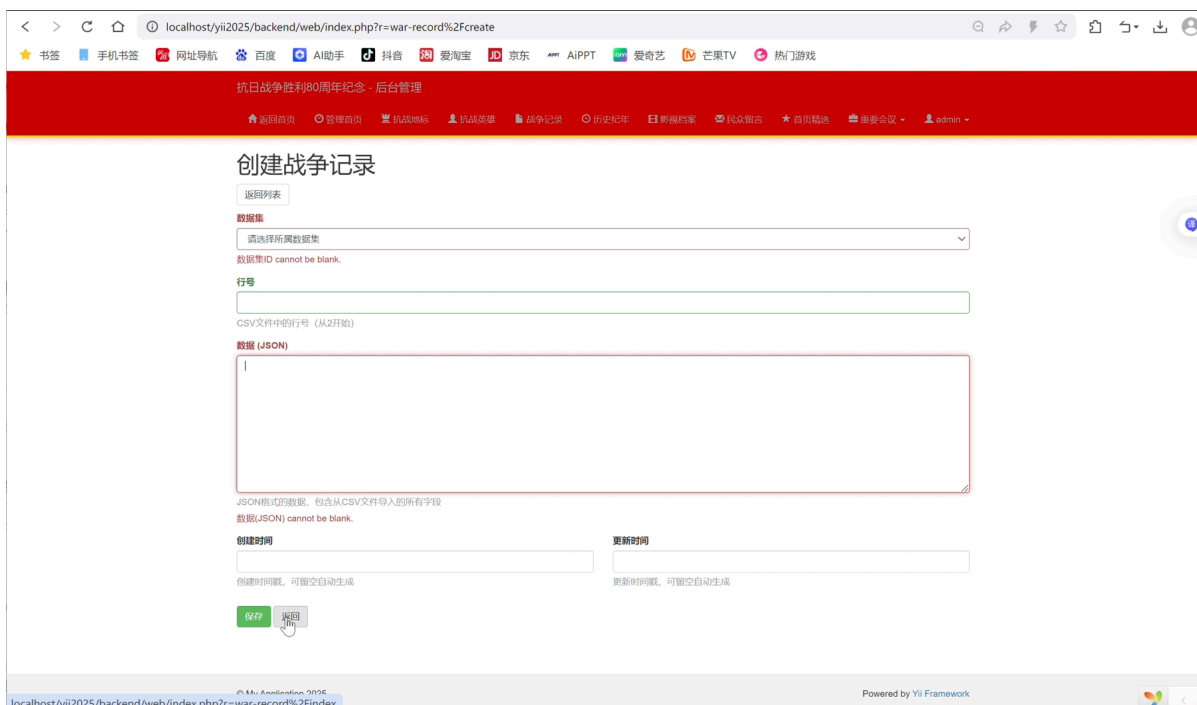
这一类记录的价值在于: 它保留了 `source/note/type`, 前台展示时可以把“口径提示/来源提示”挂出来, 而不是只丢一个数字。

效果如下:

The screenshot shows a web application interface for 'War Records' (战争记录). The interface includes a search bar, a table of records, and a sidebar with navigation links.

The table displays the following data:

#	ID	行号	数据集	数据 (JSON)	创建时间	更新时间
1	198	2	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025 1:27:32 PM	Dec 22, 2025 2:46:39 PM
2	199	3	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025 1:27:32 PM	Dec 22, 2025 1:27:32 PM
3	200	4	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025 1:27:32 PM	Dec 22, 2025 1:27:32 PM
4	201	5	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025 1:27:32 PM	Dec 22, 2025 1:27:32 PM
5	202	6	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025 1:27:32 PM	Dec 22, 2025 1:27:32 PM
6	203	7	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025 1:27:32 PM	Dec 22, 2025 1:27:32 PM
7	204	8	01_china_casualties_estimates	dataset: 中国伤亡与损失估计 (多来源); period_start: 193...	Dec 22, 2025	Dec 22, 2025



3) 前台实现

- 控制器: `frontend/controllers/wardataController.php`

核心思路是两步:

- 先查 `war_dataset`: 拿到“要展示哪些数据集 + 顺序”
- 再按 `dataset_id` 查 `war_record`: 把 `data_json` decode 成数组, 交给前端渲染

```
<?php
namespace frontend\controllers;

use yii\web\Controller;
use common\models\WarDataset;
use common\models\WarRecord;

class WardataController extends Controller
{
    public function actionIndex()
    {
        $datasets = WarDataset::find()
            ->where(['is_active' => 1])
            ->orderBy(['display_order' => SORT_ASC])
            ->all();

        return $this->render('index', [
            'datasets' => $datasets,
        ]);
    }

    public function actionView($id)
    {
        $dataset = WarDataset::findOne($id);
        if (!$dataset || !$dataset->is_active) {
            throw new \yii\web\NotFoundException('数据集不存在或已下架');
        }
    }
}
```

```

$records = warRecord::find()
    ->where(['dataset_id' => $id])
    ->orderBy(['row_index' => SORT_ASC])
    ->all();

// 把 data_json 解出来给前端用（失败则给空数组，避免页面炸）
$rows = [];
foreach ($records as $r) {
    $json = json_decode($r->data_json, true);
    $rows[] = is_array($json) ? $json : [];
}

return $this->render('view', [
    'dataset' => $dataset,
    'rows' => $rows,
]);
}
}

```

- 视图：`frontend/views/wardata/index.php`、`frontend/views/wardata/view.php`

最终效果如下：



7. 人物专栏模块

1) 数据库与模型

- 表：`figure`
- 模型：`common/models/Figure.php`

关键字段：`name` / `biography` / `achievements` / `cover_image_url` / `created_at` / `updated_at` / `deleted_at`。

人物这种内容一般更需要“下架但保留”，所以用 `deleted_at` 做软删更合适。

2) 后台实现

- 控制器: `backend/controllers/FigureController.php`

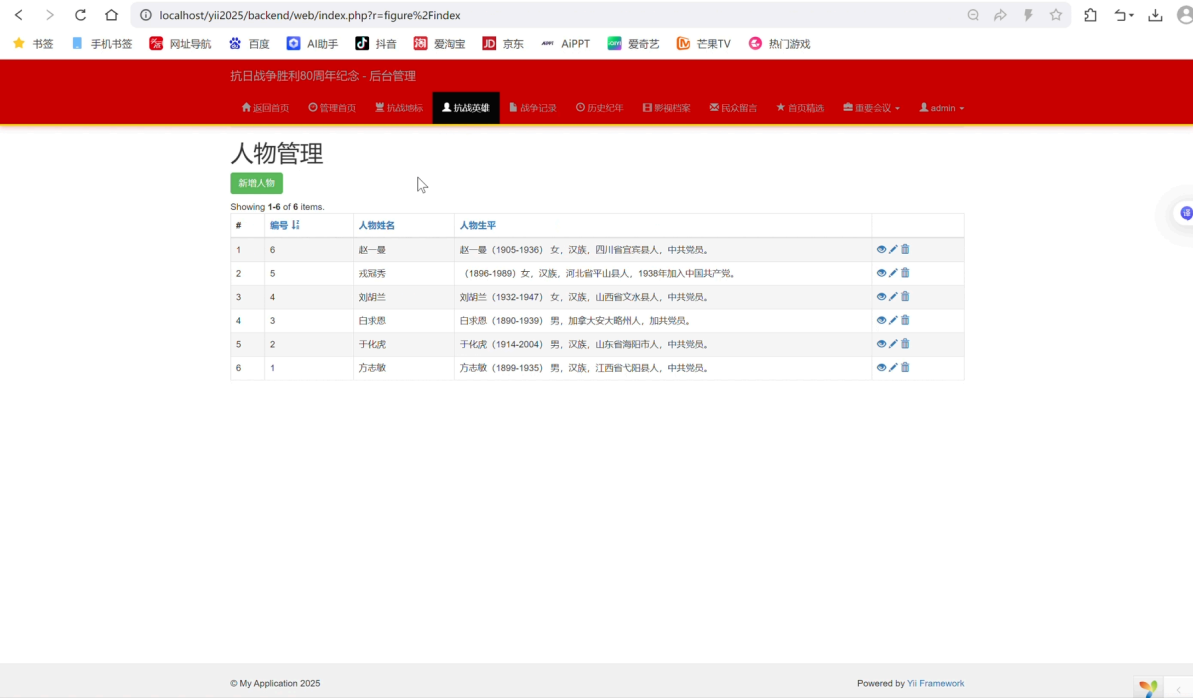
删除动作不做物理删除, 而是写 `deleted_at`, 并在查询时过滤 `NULL`:

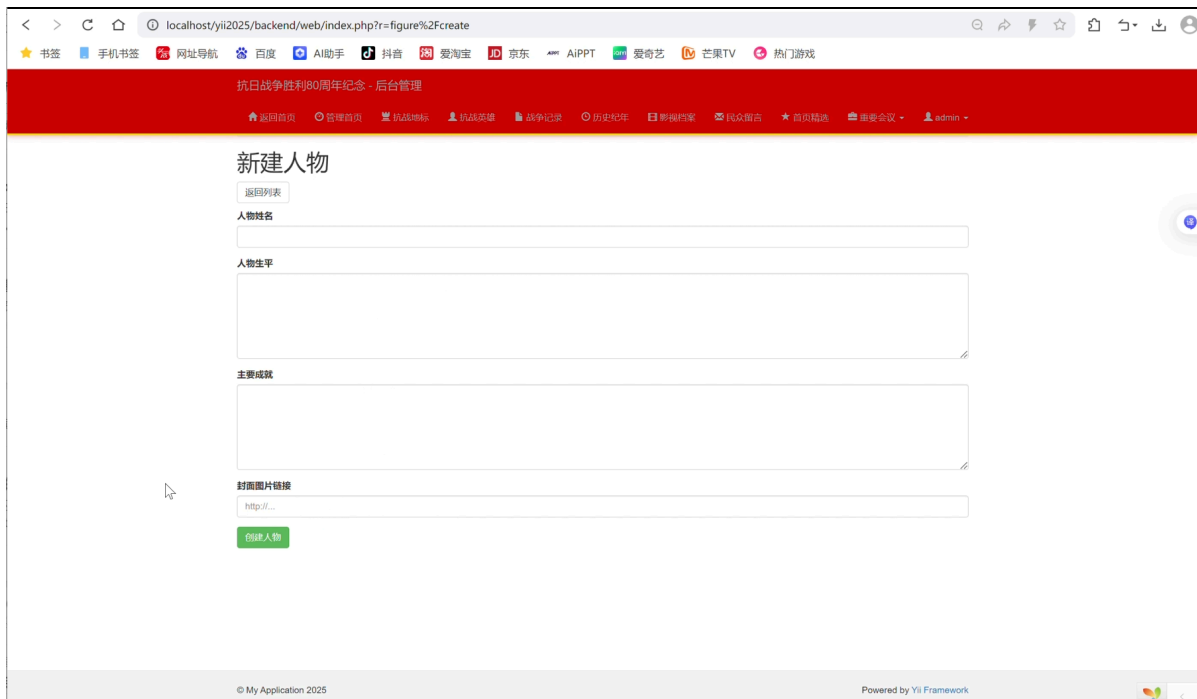
```
public function actionDelete($id)
{
    $model = $this->findModel($id);
    $model->deleted_at = time();
    $model->save(false);
    return $this->redirect(['index']);
}

protected function findModel($id)
{
    $model = Figure::find()
        ->where(['id' => $id, 'deleted_at' => null])
        ->one();

    if ($model !== null) return $model;
    throw new \yii\web\NotFoundException('人物不存在或已下架');
}
```

效果如下:





3) 前台实现

- 控制器: `frontend/controllers/FigureController.php`

```
<?php
namespace frontend\controllers;

use yii\web\Controller;
use yii\data\ActiveDataProvider;
use common\models\Figure;

class FigureController extends Controller
{
    public function actionIndex()
    {
        $query = Figure::find()->where(['deleted_at' => null]);

        $dataProvider = new ActiveDataProvider([
            'query' => $query->orderBy(['created_at' => SORT_DESC]),
            'pagination' => ['pageSize' => 12],
        ]);

        return $this->render('index', ['dataProvider' => $dataProvider]);
    }

    public function actionView($id)
    {
        $model = Figure::find()->where(['id' => $id, 'deleted_at' => null])->one();
        if (!$model) {
            throw new \yii\web\NotFoundException('人物不存在或已下架');
        }
        return $this->render('view', ['model' => $model]);
    }
}
```

- 视图: `frontend/views/figure/index.php`、`frontend/views/figure/view.php`

列表图固定卡片高度 + `object-fit: cover`，不然一堆封面高低不齐会很乱。

最终效果如下：



7. 影视资源模块

1) 数据库与模型

- 表: `media_resource`
- 模型: `common/models/MediaResource.php`

关键字段: `title / type / description / path / url / linkable_type / linkable_id / created_at / updated_at`。

我们把“封面”和“跳转链接”拆开：

- `path`：封面图（直链或站内相对路径）
- `url`：真正要跳的资源地址（播放页/专题页/下载页）

2) 模型兜底

- 文件: `common/models/MediaResource.php`

```
<?php
namespace common\models;

use yii\db\ActiveRecord;
use Yii;

class MediaResource extends ActiveRecord
{
    public static function tableName()
    {
        return 'media_resource';
    }

    public function getCoverUrl(): string
    {
        $candidate = $this->path ?: $this->url;
        if (!$candidate) {
            return Yii::getAlias('@web') . '/images/placeholder.png';
        }

        // 相对路径补全为可访问 URL
        if (!preg_match('#^https?://#i', $candidate) && strpos($candidate, '/')
        !== 0) {
            return Yii::getAlias('@web') . '/' . ltrim($candidate, '/');
        }
        return $candidate;
    }
}
```

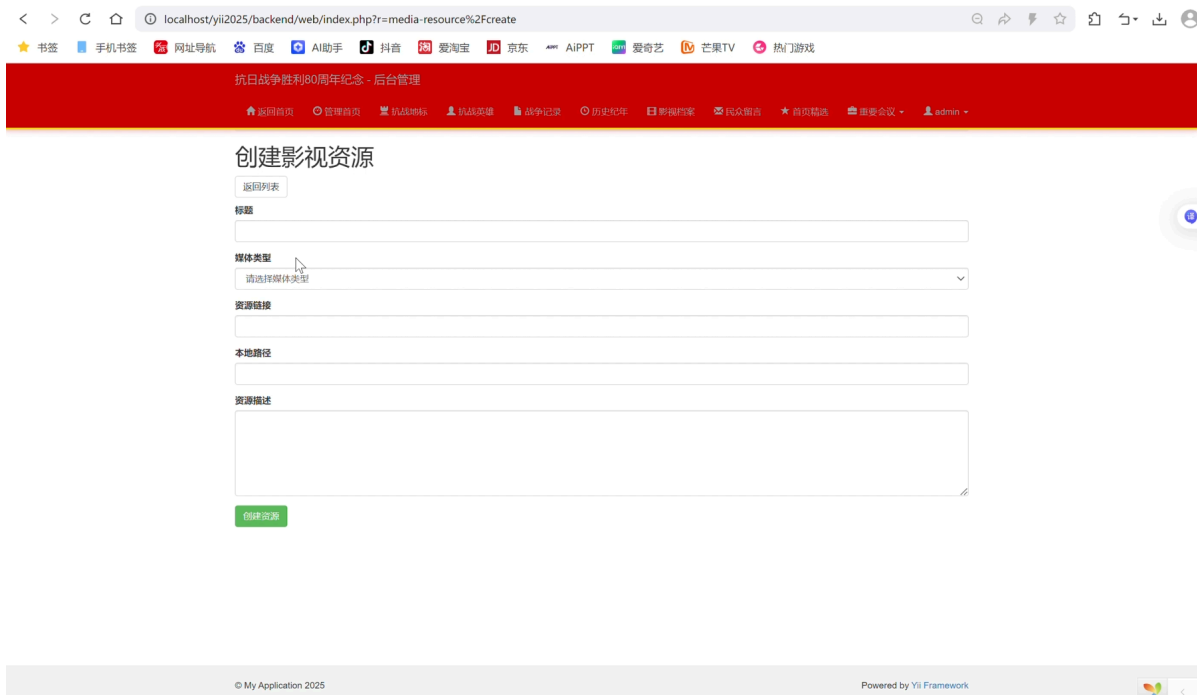
后台效果如下:

媒体资源管理

Showing 1-4 of 4 items.

#	编号	标题	类型	链接	路径
1	1	“她”没提供，已无法从医学上解释。一位生命最后遭遇了什么？	5	https://www.bilibili.com/video/BV1EG4y1x7qv?share_source=copy_web&vd_source=ef92eb37db7b3b4286ce578859043186	https://youke2.picui.cn/s/1/2025/12/16/6940defee5927.png
2	2	还记得白求恩吗？他远比课本记载的伟大大，必将被中国人永远铭记！	5	https://www.bilibili.com/video/BV15L411G7dC/?share_source=copy_web&vd_source=ef92eb37db7b3b4286ce578859043186	https://youke2.picui.cn/s/1/2025/12/16/6940def43922.png
3	9	白求恩大夫	2	https://www.bilibili.com/bangumi/play/ep313058/?share_source=copy_web	https://youke2.picui.cn/s/1/2025/12/16/6940def297c.png
4	10	南京照相馆	4	https://www.mgtv.com/h/768696.html	https://youke2.picui.cn/s/1/2025/12/16/6940df108dec.png

Powered by Yii Framework



3) 前台实现

- 控制器: `frontend/controllers/MediaController.php`

```
<?php
namespace frontend\controllers;

use yii\web\Controller;
use yii\data\ActiveDataProvider;
use common\models\MediaResource;

class MediaController extends Controller
{
    public function actionIndex($type = null)
    {
        $query = MediaResource::find();

        if ($type !== null && $type !== '') {
            $query->andWhere(['type' => (int)$type]);
        }

        $dataProvider = new ActiveDataProvider([
            'query' => $query->orderBy(['created_at' => SORT_DESC]),
            'pagination' => ['pageSize' => 12],
        ]);

        return $this->render('index', [
            'dataProvider' => $dataProvider,
            'type' => $type,
        ]);
    }

    public function actionView($id)
    {
        $model = MediaResource::findOne($id);
        if (!$model) {
```

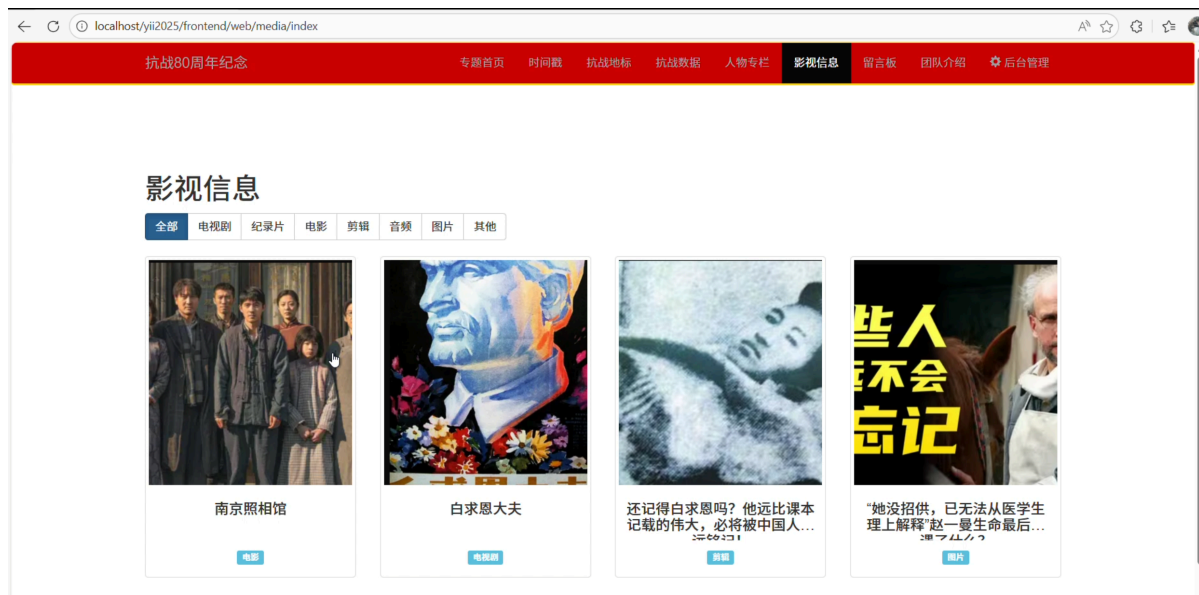
```

        throw new \yii\web\NotFoundException('资源不存在');
    }
    return $this->render('view', ['model' => $model]);
}
}

```

列表用 `getCoverUrl()` 画封面，详情页按钮用 `url` 做跳转，这样“裂图”“点不开”的概率会明显低很多。

最终效果如下：



8. 留言板模块

1) Gii 生成 CRUD

生成 Model: `http://localhost/yii2025/gii/model`

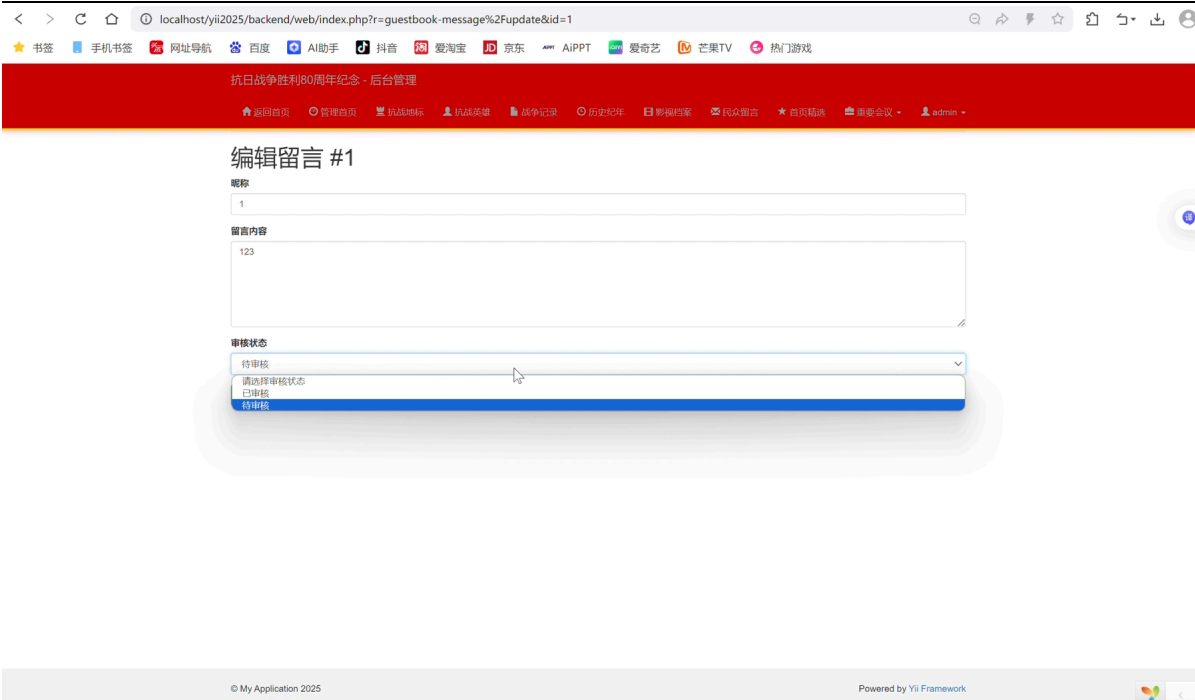
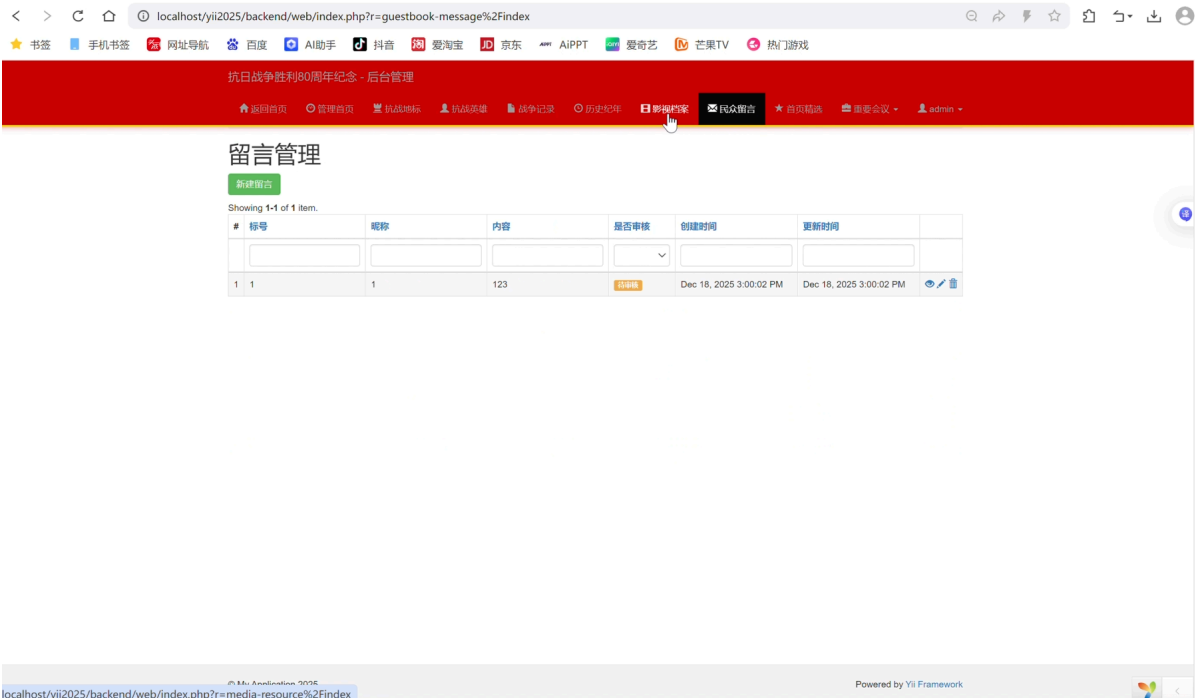
- Table Name: `guestbook_message`
- Model Class: `GuestbookMessage`

- Namespace: `common\models`

生成 CRUD (后台) : `http://localhost/yii2025/gii/crud`

- Model Class: `common\models\GuestbookMessage`
- Search Model Class: `backend\models\GuestbookMessageSearch`
- Controller Class: `backend\controllers\GuestbookMessageController`
- View Path: `@backend/views/guestbook-message`

后台管理效果如下:



2) 数据库建表

```
CREATE TABLE `guestbook_message` (  
    `id` INT NOT NULL AUTO_INCREMENT COMMENT '主键ID',  
    `nickname` VARCHAR(100) NOT NULL DEFAULT '' COMMENT '昵称',  
    `content` TEXT NOT NULL COMMENT '留言内容',  
    `is_approved` TINYINT(1) NOT NULL DEFAULT '0' COMMENT '是否审核通过: 0-待审核, 1-已通过',  
    `created_at` INT(11) NOT NULL DEFAULT '0' COMMENT '创建时间戳',  
    `updated_at` INT(11) NOT NULL DEFAULT '0' COMMENT '更新时间戳',  
    PRIMARY KEY (`id`),  
    KEY `idx_is_approved` (`is_approved`),  
    KEY `idx_created_at` (`created_at`)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='留言表';
```

3) Model 实现

- 文件: `common/models/GuestbookMessage.php`

```
<?php  
  
namespace common\models;  
  
use Yii;  
use yii\behaviors\TimestampBehavior;  
  
class GuestbookMessage extends \yii\db\ActiveRecord  
{  
    public static function tableName()  
    {  
        return 'guestbook_message';  
    }  
  
    public function behaviors()  
    {  
        return [  
            TimestampBehavior::class,  
        ];  
    }  
  
    public function rules()  
    {  
        return [  
            [['content'], 'required'],  
            [['content'], 'string'],  
            [['nickname'], 'string', 'max' => 100],  
            [['is_approved'], 'boolean'],  
            [['created_at', 'updated_at'], 'integer'],  
        ];  
    }  
  
    public function attributeLabels()  
    {  

```

```

        return [
            'id' => 'ID',
            'nickname' => '昵称',
            'content' => '留言内容',
            'is_approved' => '审核状态',
            'created_at' => '创建时间',
            'updated_at' => '更新时间',
        ];
    }
}

```

4) 前台 Controller

- 文件: `frontend/controllers/GuestbookController.php`

```

<?php
namespace frontend\controllers;

use Yii;
use yii\web\Controller;
use common\models\GuestbookMessage;

class GuestbookController extends Controller
{
    public function actionIndex()
    {
        $model = new GuestbookMessage();

        if ($model->load(Yii::$app->request->post())) {
            $model->is_approved = 0; // 新留言默认未审核

            if ($model->save()) {
                Yii::$app->session->setFlash('success', '您的留言已提交，请等待管理员
审核。');
                return $this->refresh(); // 防止刷新重复提交
            }
        }

        $messages = GuestbookMessage::find()
            ->where(['is_approved' => 1]) // 只显示已审核
            ->orderBy(['created_at' => SORT_DESC])
            ->all();

        return $this->render('index', [
            'messages' => $messages,
            'model' => $model,
        ]);
    }
}

```

5) 后台 Controller

- 文件: backend/controllers/GuestbookMessageController.php

```
<?php

namespace backend\controllers;

use Yii;
use common\models\GuestbookMessage;
use backend\models\GuestbookMessageSearch;
use yii\web\Controller;
use yii\web\NotFoundHttpException;
use yii\filters\VerbFilter;
use yii\filters\AccessControl;

class GuestbookMessageController extends Controller
{
    public function behaviors()
    {
        return [
            'access' => [
                'class' => AccessControl::class,
                'rules' => [
                    ['allow' => true, 'roles' => ['@']],
                ],
            ],
            'verbs' => [
                'class' => VerbFilter::class,
                'actions' => [
                    'delete' => ['POST'],
                    'approve' => ['POST'],
                ],
            ],
        ];
    }

    public function actionIndex()
    {
        $searchModel = new GuestbookMessageSearch();
        $dataProvider = $searchModel->search(Yii::$app->request->queryParams);

        return $this->render('index', [
            'searchModel' => $searchModel,
            'dataProvider' => $dataProvider,
        ]);
    }

    public function actionView($id)
    {
        return $this->render('view', [
            'model' => $this->findModel($id),
        ]);
    }
}
```

```

public function createAction()
{
    $model = new GuestbookMessage();

    if ($model->load(Yii::$app->request->post()) && $model->save()) {
        return $this->redirect(['view', 'id' => $model->id]);
    }

    return $this->render('create', ['model' => $model]);
}

public function actionUpdate($id)
{
    $model = $this->findModel($id);

    if ($model->load(Yii::$app->request->post()) && $model->save()) {
        return $this->redirect(['view', 'id' => $model->id]);
    }

    return $this->render('update', ['model' => $model]);
}

public function actionDelete($id)
{
    $model = $this->findModel($id);

    if ($model->delete()) {
        Yii::$app->session->setFlash('success', '留言已删除');
    } else {
        Yii::$app->session->setFlash('error', '删除失败');
    }

    return $this->redirect(['index']);
}

public function actionApprove($id)
{
    $model = $this->findModel($id);

    $model->is_approved = $model->is_approved ? 0 : 1;

    if ($model->save(false)) {
        Yii::$app->session->setFlash('success', $model->is_approved ? '留言已
审核通过' : '留言审核已撤销');
    } else {
        Yii::$app->session->setFlash('error', '操作失败');
    }

    return $this->redirect(['index']);
}

protected function findModel($id)
{
    if (($model = GuestbookMessage::findOne($id)) !== null) {
        return $model;
    }
}

```

```
        throw new NotFoundException('页面不存在');
    }
}
```

6) 前台视图

- 文件: `frontend/views/guestbook/index.php`

```
<?php
use yii\helpers\Html;
use yii\widgets\ActiveForm;

$this->title = '留言板';
$this->params['breadcrumbs'][] = $this->title;
?>

<style>
.guestbook-form { border:1px solid #ddd; border-radius:8px; padding:20px; margin-
bottom:30px; }
.messages-section { border:1px solid #ddd; border-radius:8px; padding:20px; }
.messages-section h3 { margin-top:0; margin-bottom:20px; padding-bottom:10px;
border-bottom:1px solid #eee; }
.message-item { border:1px solid #e0e0e0; border-radius:6px; padding:15px;
margin-bottom:12px; }
.message-header { display:flex; justify-content:space-between; margin-
bottom:10px; font-size:0.9rem; }
.message-author { font-weight:600; color:#333; }
.message-time { color:#999; }
.message-content { color:#555; line-height:1.6; }
.empty-message { text-align:center; color:#999; padding:30px; }
</style>

<div class="guestbook-index">
    <h1><?= Html::encode($this->title) ?></h1>

    <div class="guestbook-form">
        <?php $form = ActiveForm::begin(); ?>

        <?= $form->field($model, 'nickname')->textInput(['maxlength' => true]) ?>
        <?= $form->field($model, 'content')->textarea(['rows' => 5]) ?>

        <div class="form-group">
            <?= Html::submitButton('提交', ['class' => 'btn btn-primary']) ?>
        </div>

        <?php ActiveForm::end(); ?>
    </div>

    <div class="messages-section">
        <h3>留言列表</h3>

        <?php if (!empty($messages)): ?>
            <?php foreach ($messages as $message): ?>
                <div class="message-item">
                    <div class="message-header">
```

```

        <span class="message-author"><?= Html::encode($message-
>nickname) ?></span>
        <span class="message-time"><?= Yii::$app->formatter-
>asDatetime($message->created_at) ?></span>
    </div>
    <div class="message-content">
        <?= Html::encode($message->content) ?>
    </div>
</div>
<?php endforeach; ?>
<?php else: ?>
    <p class="empty-message">暂无留言</p>
<?php endif; ?>
</div>
</div>

```

7) 后台视图

- 文件: `backend/views/guestbook-message/index.php`

这里主要做了两件小修正: 标题引号统一、`created_at` 不直接用时间戳而是格式化显示。

```

<?php
use yii\helpers\Html;
use yii\grid\GridView;

$this->title = '留言审核界面';
$this->params['breadcrumbs'][] = $this->title;
?>

<div class="guestbook-message-index">
    <h1><?= Html::encode($this->title) ?></h1>

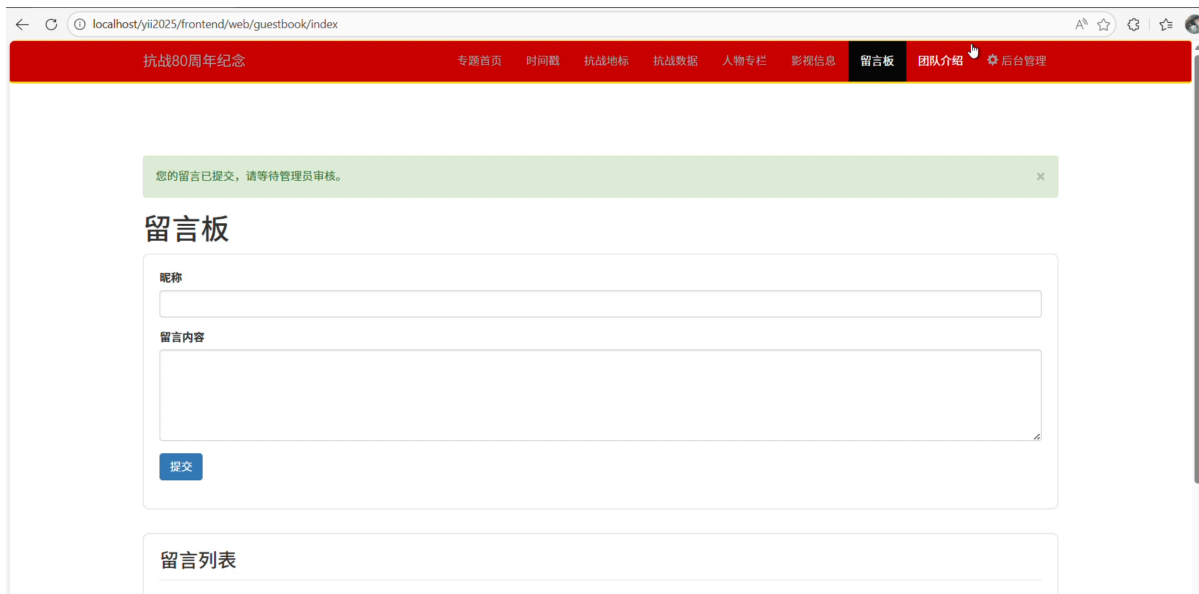
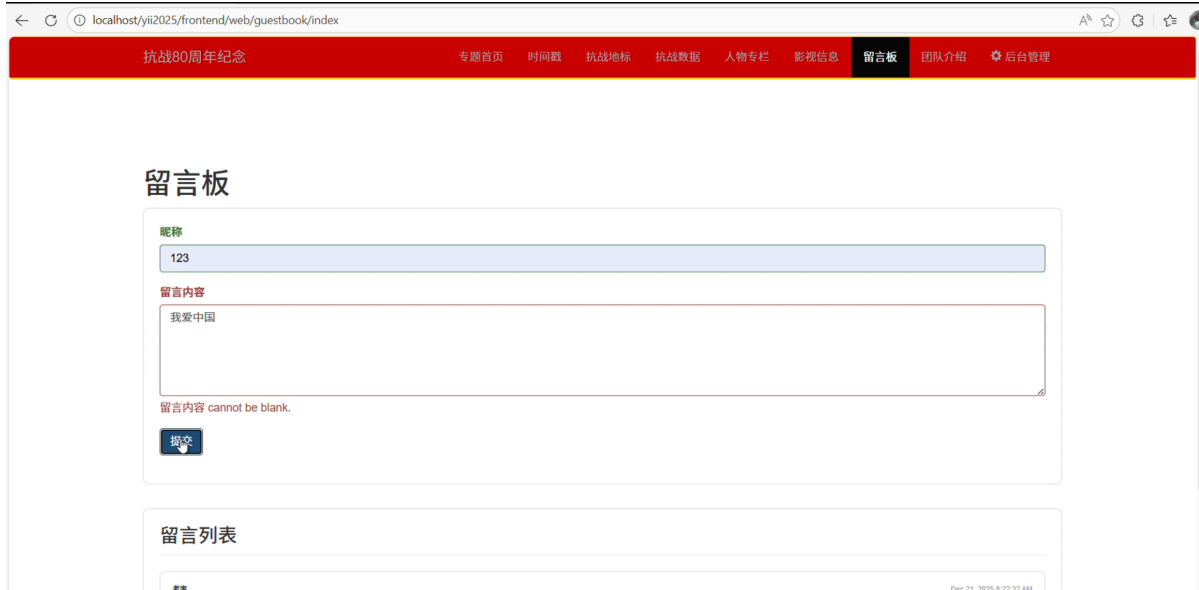
    <p><?= Html::a('创建留言', ['create'], ['class' => 'btn btn-success']) ?></p>

    <?= GridView::widget([
        'dataProvider' => $dataProvider,
        'filterModel' => $searchModel,
        'columns' => [
            ['class' => 'yii\grid\SerialColumn'],
            'id',
            'nickname',
            'content:text',
            [
                'attribute' => 'is_approved',
                'filter' => [0 => '待审核', 1 => '已通过'],
                'value' => fn($m) => $m->is_approved ? '已通过' : '待审核',
            ],
            [
                'attribute' => 'created_at',
                'value' => fn($m) => Yii::$app->formatter->asDatetime($m-
>created_at),
            ],
            [
                'class' => 'yii\grid\ActionColumn',

```

```
        'template' => '{view} {update} {delete}',
    ],
],
}); ?>
</div>
```

最终效果如下:



9. 团队介绍模块

1) 路由配置

- 文件: `frontend/config/main.php`


```
'urlManager' => [
    'enablePrettyUrl' => true,
    'showScriptName' => false,
    'rules' => [
        'team' => 'site/team',
    ],
],
```

2) Controller 实现

- 文件: `frontend/controllers/SiteController.php`

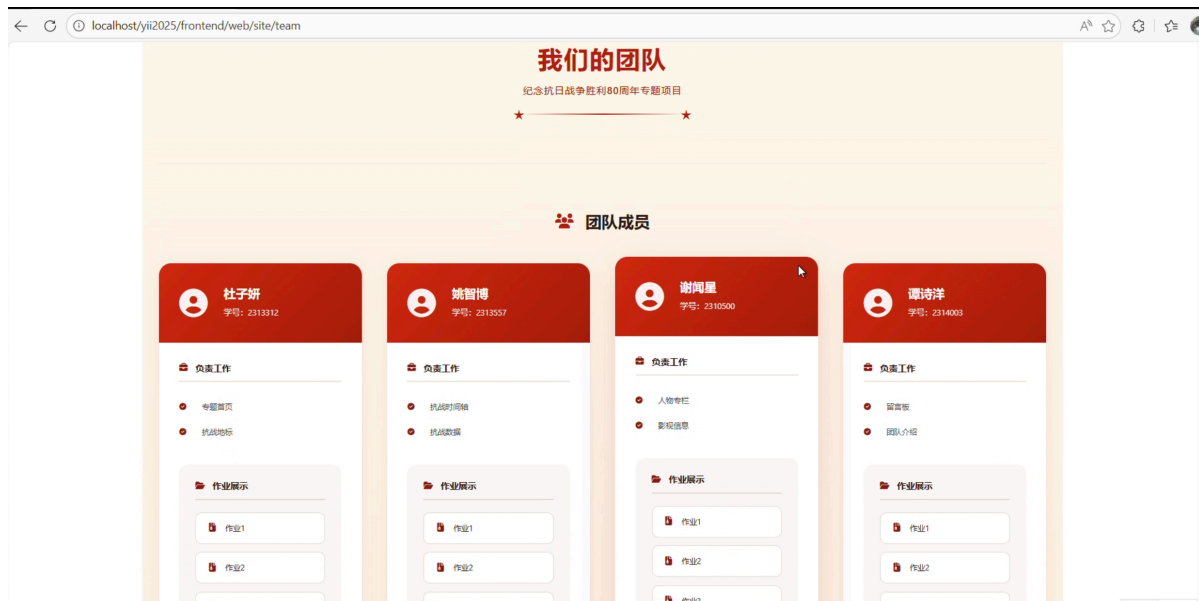
```
public function actionTeam()
{
    return $this->render('team');
}
```

3) 视图实现

- 文件: `frontend/views/site/team.php`

页面基本就是纯 HTML/CSS：标题区 + 成员卡片区 + 成果展示区。成员卡片里放头像/姓名/学号/分工/作业链接；成果区放项目主题、技术栈、进度、模块入口和资料链接，整体偏“展示型”而不是“功能型”。

最终效果如下：



10. 安全与质量保证

1) 输入与存储

- 表单数据走模型 `rules()` 校验：必填就 required，长度就限制长度
- 数据库操作用 ActiveRecord / 参数化查询，避免手拼 SQL

2) 输出与前端展示

- 用户可输入内容（尤其留言）统一用 `Html::encode()` 输出，防 XSS

3) 会话与后台权限

- 后台 controller 用 `AccessControl` 限制登录用户访问
- 删除、审核这种敏感操作用 `VerbFilter` 强制 POST，减少误触与 CSRF 风险
- Yii2 默认启用 CSRF，表单会自动带隐藏字段

4) 内容治理

- 人物用 `deleted_at` 软删：下架但不丢内容，引用关系也不容易断
- 留言用 `is_approved`：前台永远只展示通过审核的
- 首页入口用 `is_active`：不需要为了开关入口去改前端代码